

Week 4 Lab

Microservices and Cloud-Native Design

Lab Documentation Report

Name: Shahd Tamer Abdelmonem (SE3)

1. Lab Overview

This lab involved building a small cloud-native system from scratch using two microservices and Docker Compose. The system demonstrates key cloud-native concepts including 12-factor app design, microservices architecture, synchronous service-to-service communication, container packaging, health checks, and basic resilience.

2. Part A Project Structure

The following folder structure was created for the project:

```
PS C:\Users\shahd> cd C:\Users\shahd\OneDrive\Desktop\week4-lab
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> wsl
docker-desktop:/tmp/docker-desktop-root/run/desktop/mnt/host/c/Users/shahd/OneDrive/Desktop/week4-lab# touch docker-compose.yml
ts.txt product-service/Dockerfile
touch order-service/app.py order-service/requirements.txt order-service/Dockerfile
docker-desktop:/tmp/docker-desktop-root/run/desktop/mnt/host/c/Users/shahd/OneDrive/Desktop/week4-lab# touch product-service/app.py product-service/requirements.txt
touch product-service/Dockerfile
docker-desktop:/tmp/docker-desktop-root/run/desktop/mnt/host/c/Users/shahd/OneDrive/Desktop/week4-lab# touch order-service/app.py order-service/requirements.txt
touch order-service/Dockerfile
docker-desktop:/tmp/docker-desktop-root/run/desktop/mnt/host/c/Users/shahd/OneDrive/Desktop/week4-lab# |
```

3. System Architecture

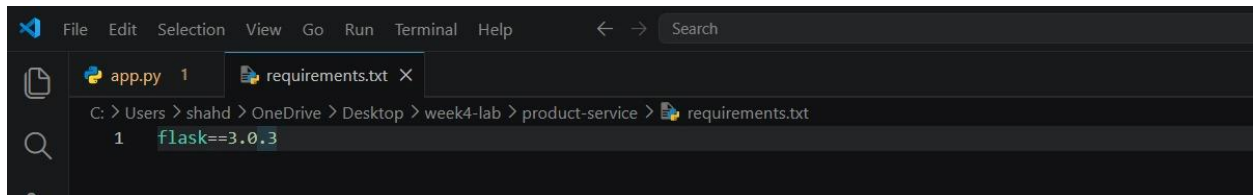
The system consists of two microservices communicating synchronously over HTTP:

Component	Technology	Port	Responsibility
product-service	Python Flask	5001	Returns product data from an in-memory catalog
order-service	Python Flask + requests	5002	Creates orders after validating with product-service
pDocker Compose	Container orchestration	N/A	Runs and manages both services

4. Part B — Product Service Implementation

The product-service was implemented using Python Flask. It exposes two endpoints: /health for monitoring and /products/<product_id> for product lookup. An in-memory dictionary was used to store the product catalog

4.1 Product-service/requirements.txt

A screenshot of a code editor window. The title bar shows 'File Edit Selection View Go Run Terminal Help'. The editor has two tabs: 'app.py 1' and 'requirements.txt X'. The 'requirements.txt' tab is active, showing the file path 'C: > Users > shahd > OneDrive > Desktop > week4-lab > product-service > requirements.txt'. The content of the file is a single line: '1 flask==3.0.3'.

4.2 Product-service/app.py

```
1 from flask import Flask, jsonify
2
3 app = Flask(__name__)
4
5 PRODUCTS = {
6     1: {"id": 1, "name": "Laptop", "price": 1200},
7     2: {"id": 2, "name": "Phone", "price": 650},
8     3: {"id": 3, "name": "Headphones", "price": 120}
9 }
10
11 @app.route("/health", methods=["GET"])
12 def health():
13     return jsonify({"service": "product-service", "status": "up"}), 200
14
15 @app.route("/products/<int:product_id>", methods=["GET"])
16 def get_product(product_id):
17     product = PRODUCTS.get(product_id)
18     if product:
19         return jsonify(product), 200
20     return jsonify({"error": "Product not found"}), 404
21
22 if __name__ == "__main__":
23     app.run(host="0.0.0.0", port=5001)
```

4.3 Key Design Points

- The service is stateless — no dependency on local mutable files.
- The /health endpoint supports container orchestration and monitoring.
- All responses are in JSON format, the standard for microservice communication.
-

5. Part C — Order Service Implementation

The order-service was implemented to receive POST requests, call product-service to validate the product, and return a calculated order total. Environment variables and retry logic were used to follow cloud-native design principles.

5.1 Order-service/requirements.txt

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays the contents of a file named requirements.txt, which lists two dependencies: flask==3.0.3 and requests==2.32.3. The lines are numbered 1, 2, and 3 respectively.

```
1 flask==3.0.3
2 requests==2.32.3
3
```

Create order-service/app.py

```
1 import os
2 import time
3 import requests
4 from flask import Flask, jsonify, request
5
6 app = Flask(__name__)
7
8 PRODUCT_SERVICE_URL = os.getenv("PRODUCT_SERVICE_URL", "http://product-service:5001")
9
10 def fetch_product(product_id, retries=2, delay=1):
11     url = f"{PRODUCT_SERVICE_URL}/products/{product_id}"
12     for attempt in range(retries + 1):
13         try:
14             response = requests.get(url, timeout=2)
15             return response
16         except requests.exceptions.RequestException:
17             if attempt < retries:
18                 time.sleep(delay)
19             else:
20                 return None
21
22 @app.route("/health", methods=["GET"])
23 def health():
24     return jsonify({"service": "order-service", "status": "up"}), 200
25
26 @app.route("/orders", methods=["POST"])
27 def create_order():
28     data = request.get_json()
29     product_id = data.get("product_id")
30     quantity = data.get("quantity", 1)
31
32     response = fetch_product(product_id)
33
34     if response is None:
35         return jsonify({"error": "product-service unavailable"}), 503
36     if response.status_code != 200:
37         return jsonify({"error": "invalid product"}), 400
38
39     product = response.json()
40     total = product["price"] * quantity
41
42     return jsonify({
43         "message": "Order created",
44         "product": product["name"],
45         "quantity": quantity,
46         "total_price": total
47     }), 201
48
49 if __name__ == "__main__":
50     app.run(host="0.0.0.0", port=5002)
```

5.2 Key Implementation Details

- `PRODUCT_SERVICE_URL` is read from an environment variable, not hardcoded.
- The `fetch_product()` function retries up to 2 times with a 1-second delay.
- A 2-second timeout prevents the service from hanging on slow responses.
- Returns HTTP 503 if product-service is unreachable, 400 for invalid products.

6. Part D — Containerization

Each service was packaged with its own Dockerfile using the `python:3.12-slim` base image to keep images small and production-ready.

6.1 Product-service/Dockerfile



```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 EXPOSE 5001
7 CMD ["python", "app.py"]
```

6.2 Order-service/Dockerfile



```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 EXPOSE 5002
7 CMD ["python", "app.py"]
```

6.2 Design Decisions

- requirements.txt is copied before app.py to leverage Docker layer caching.
- Each service has its own independent image, supporting independent deployment.
- The slim base image reduces attack surface and image size.

7. Part E — Docker Compose Orchestration

The docker-compose.yml file was created to define both services, their environment variables, health checks, port mappings, and restart policies:

Create docker-compose.yml

```
1  services:
2    product-service:
3      build: ./product-service
4      container_name: product-service
5      ports:
6        - "5001:5001"
7      healthcheck:
8        test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5001/health')"]
9        interval: 10s
10       timeout: 5s
11       retries: 3
12       restart: unless-stopped
13
14    order-service:
15      build: ./order-service
16      container_name: order-service
17      ports:
18        - "5002:5002"
19      environment:
20        PRODUCT_SERVICE_URL: http://product-service:5001
21      depends_on:
22        - product-service
23      healthcheck:
24        test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5002/health')"]
25        interval: 10s
26        timeout: 5s
27        retries: 3
28       restart: unless-stopped
```

7.1 Key Configuration Points

- Environment variable PRODUCT_SERVICE_URL is injected into order-service.
- depends_on ensures product-service starts before order-service.
- Health checks run every 10 seconds to monitor service availability.
- restart: unless-stopped provides first-level resilience.
- Docker's internal DNS resolves service names (e.g., product-service) automatically.

8. Part F — Build and Run Evidence

F1 — Building and Starting the System

From inside the week4-lab/ folder, the following command was run to build the Docker images and start both containers in detached mode:

```
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> docker compose up
[+] up 5/5
✓Image week4-lab-order-service Built 16.9s
✓Image week4-lab-product-service Built 16.9s
✓Network week4-lab_default Created 0.1s
✓Container product-service Started 1.5s
✓Container order-service Started 1.4s
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> docker compose ps
NAME                IMAGE                COMMAND              SERVICE      CREATED      STATUS      PORTS
order-service       week4-lab-order-service "python app.py"      order-service About a minute ago Up About a minute (healthy) 0.0.0.0:5002->5002/tcp,
[:]:5002->5002/tcp
product-service     week4-lab-product-service "python app.py"      product-service About a minute ago Up About a minute (healthy) 0.0.0.0:5001->5001/tcp,
[:]:5001->5001/tcp
PS C:\Users\shahd\OneDrive\Desktop\week4-lab>
```

Both containers started successfully with ports 5001 and 5002 exposed on the host machine.

F2 — Service Logs Verification

Logs were checked for both services to confirm they started correctly:

```
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> docker compose logs product-service
product-service | * Serving Flask app 'app'
product-service | * Debug mode: off
product-service | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
product-service | * Running on all addresses (0.0.0.0)
product-service | * Running on http://127.0.0.1:5001
product-service | * Running on http://172.19.0.2:5001
product-service | Press CTRL+C to quit
product-service | 127.0.0.1 - - [06/May/2026 00:20:52] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:21:02] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:21:12] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:21:23] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:21:33] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:21:43] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:21:53] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:22:03] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:22:13] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:22:23] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:22:33] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:22:44] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:22:54] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:23:04] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:23:14] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:23:24] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:23:34] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:23:45] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:23:55] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:24:05] "GET /health HTTP/1.1" 200 -
product-service | 127.0.0.1 - - [06/May/2026 00:24:15] "GET /health HTTP/1.1" 200 -
```

```

PS C:\Users\shahd\OneDrive\Desktop\week4-lab> docker compose logs order-service
order-service | * Serving Flask app 'app'
order-service | * Debug mode: off
order-service | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
order-service | * Running on all addresses (0.0.0.0)
order-service | * Running on http://127.0.0.1:5002
order-service | * Running on http://172.19.0.3:5002
order-service | Press CTRL+C to quit
order-service | 127.0.0.1 - - [06/May/2026 00:20:52] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:21:03] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:21:13] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:21:23] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:21:33] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:21:43] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:21:53] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:22:03] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:22:14] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:22:24] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:22:34] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:22:44] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:22:54] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:23:04] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:23:14] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:23:24] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:23:35] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:23:45] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:23:55] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:24:05] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:24:15] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:24:25] "GET /health HTTP/1.1" 200 -
order-service | 127.0.0.1 - - [06/May/2026 00:24:35] "GET /health HTTP/1.1" 200 -
PS C:\Users\shahd\OneDrive\Desktop\week4-lab>

```

Observation: Both services started successfully. Docker's health check system was automatically calling `/health` on each service every 10 seconds and receiving HTTP 200 responses, confirming both services were healthy.

9. Part G — Functional Testing Evidence

G1 — Product Service Tests

The product-service was tested directly using `Invoke-WebRequest` in PowerShell. Note: PowerShell's built-in `curl` is an alias for `Invoke-WebRequest` and does not support Linux `curl` syntax, so `Invoke-WebRequest` was used instead.

Everything is working perfectly! ✅ Here's what each result means:

Results Explained

Test	Result	Meaning
<code>/health</code>	<code>"status": "up" + 200</code>	✅ Service is running
<code>/products/1</code>	<code>{"id":1,"name":"Laptop","price":1200} + 200</code>	✅ Product found correctly
<code>/products/99</code>	<code>{"error":"Product not found"} + 404</code>	✅ Correct error handling

The red error for `/products/99` is not a problem — PowerShell shows 404 responses as errors, but that's exactly the behavior the lab expects!

```

StatusCode      : 200
StatusDescription : OK
Content         : {"service":"product-service","status":"up"}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 44
                  Content-Type: application/json
                  Date: Wed, 06 May 2026 00:29:47 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 44], [Content-Type, application/json], [Date, Wed, 06 May 2026 00:29:47 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 44

```

```

StatusCode      : 200
StatusDescription : OK
Content         : {"id":1,"name":"Laptop","price":1200}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 38
                  Content-Type: application/json
                  Date: Wed, 06 May 2026 00:29:53 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 38], [Content-Type, application/json], [Date, Wed, 06 May 2026 00:29:53 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 38

```

Command	Expected Result	Actual Result
GET /health	200 + status: up	200 OK — {"service":"product-service","status":"up"}
GET /products/1	200 + Laptop data	200 OK — {"id":1,"name":"Laptop","price":1200}
GET /products/99	404 + error message	404 — {"error":"Product not found"}

G2 — Order Service Test

The order-service was tested by sending a POST request. Because Windows PowerShell does not support the Linux curl -X POST syntax with backslash line continuation, Invoke-WebRequest was used:

```

curl : {"error":"Product not found"}
At line:1 char:1
+ curl http://localhost:5001/products/99

```

Response received:

```
StatusCode      : 201
StatusDescription : CREATED
Content         : {"message":"Order created","product":"Laptop","quantity":2,"total_price":2400}

RawContent      : HTTP/1.1 201 CREATED
                  Connection: close
                  Content-Length: 79
                  Content-Type: application/json
                  Date: Wed, 06 May 2026 00:38:01 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13

                  {"message":"Order created","p...
Forms           : {}
Headers         : {[Connection, close], [Content-Length, 79], [Content-Type, application/json], [Date, Wed, 06 May 2026 00:38:01 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 79
```

Observation: The order-service successfully called product-service internally, retrieved the Laptop product at price 1200, multiplied by quantity 2, and returned the correct total of 2400.

10. Part H — Failure Injection and Resilience Evidence

H1 — Stopping product-service

product-service was deliberately stopped to simulate a microservice failure:

```
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> docker stop product-service
product-service
```

H2 — Sending Order Request with product-service Down

```
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> Invoke-WebRequest -Uri http://localhost:5002/orders -Method POST -Headers @{"Content-Type"="application/json"}
-Body '{"product_id": 2, "quantity": 1}'
Invoke-WebRequest : {"error":"product-service unavailable"}
At line:1 char:1
+ Invoke-WebRequest -Uri http://localhost:5002/orders -Method POST -Hea ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
PS C:\Users\shahd\OneDrive\Desktop\week4-lab>
```

H3 — Restarting product-service

product-service was restarted and the system was verified to have recovered:

```
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> docker start product-service
product-service
PS C:\Users\shahd\OneDrive\Desktop\week4-lab> docker compose ps
NAME                IMAGE              COMMAND              SERVICE      CREATED      STATUS      PORTS
order-service       week4-lab-order-service  "python app.py"      order-service  22 minutes ago  Up 22 minutes (healthy)  0.0.0.0:5002->
product-service     week4-lab-product-service  "python app.py"      product-service  22 minutes ago  Up Less than a second (health: starting)  0.0.0.0:5001->
5001/tcp, [::]:5001->5001/tcp
PS C:\Users\shahd\OneDrive\Desktop\week4-lab>
```

Output confirmed both containers returned to running status. A follow-up order request after recovery returned HTTP 201 successfully, confirming full system recovery.

State	Request	Response	HTTP Status
Both services running	POST /orders {product_id:1, quantity:2}	Order created — total_price: 2400	201 Created
product-service stopped	POST /orders {product_id:2, quantity:1}	product-service unavailable	503 Service Unavailable
product-service restarted	POST /orders {product_id:1, quantity:2}	Order created — total_price: 2400	201 Created

Takeaway: Synchronous microservice communication means that when product-service is unavailable, order-service cannot complete requests. The retry logic and timeout in `fetch_product()` give the system a small resilience window, but ultimately the failure propagates. Docker Compose restart policies help recover from crashes automatically, but they do not protect against the window of unavailability during restart.

11. Part I — Reflection Questions

1. Which parts of this lab show the benefits of microservices over a monolith?

Each service has its own Dockerfile, requirements file, and codebase, meaning they can be built, updated, and deployed independently without affecting the other. For example, product-service could be updated or scaled without redeploying order-service. In a monolith, all components would share a single deployment unit, making independent updates impossible. The separation also meant each service only installs the dependencies it needs — order-service needs the requests library while product-service does not.

2. Which new complexities were introduced by splitting the system into two services?

Several complexities that do not exist in a monolith were introduced. Network calls between services can fail, time out, or be slow, which required implementing retry logic and timeout handling in `fetch_product()`. Service discovery was needed — Docker Compose resolved service names automatically via internal DNS, but this requires explicit configuration through the `PRODUCT_SERVICE_URL` environment variable. Configuration management also became more complex: instead of a direct function call, the address of product-service had to be passed to order-service via an environment variable defined in `docker-compose.yml`.

3. What would break if network latency increased or one service became slow?

If network latency increased, order-service requests would take longer because each order requires a synchronous HTTP call to product-service. If latency exceeded the 2-second timeout, the request would fail and retry, adding up to 6 seconds of total wait time per order request. Under high load, slow responses from product-service would cause order-service threads to back up, potentially making order-service appear unresponsive even though the problem originates in product-service. This is called cascading failure and is a key risk in synchronous microservice architectures. An asynchronous or queue-based design would reduce this risk.

4. Which 12-factor app principles are visible in this implementation?

Factor	Implementation in This Lab
III. Config	PRODUCT_SERVICE_URL is stored in environment variables defined in docker-compose.yml, not hardcoded in source code
VI. Processes	Both services are stateless. Product data is stored in an in-memory dictionary with no local file state or session dependency
VII. Port Binding	Each service exports itself on a specific port (5001 and 5002) declared in its Dockerfile and Compose configuration
IX. Disposability	restart: unless-stopped in docker-compose.yml enables fast recovery when a container crashes
X. Dev/Prod Parity	Docker containers ensure the same runtime environment runs locally and would run in any cloud environment

12. Conclusion

This lab successfully demonstrated the implementation of a cloud-native microservices system using Python Flask and Docker Compose. Both services were containerized, orchestrated, and tested including failure simulation. The hands-on experience showed the practical benefits of microservices such as independent deployment and isolated concerns, while also revealing the real complexities introduced by distributed systems including network failures, configuration management, and cascading failures. The 12-factor app principles proved to be a practical guide for building services that are portable, resilient, and operable in cloud environments.